Scaling the Firmware Upgrade Controller to 5–15 Million Devices

1. Challenges and Considerations

The current controller successfully demonstrates the firmware upgrade workflow with a limited number of devices (e.g. 1,000). However, scaling to 5, 10, or 15 million devices introduces additional challenges that require architectural changes, including:

- **Resource limitations:** CPU, memory (RAM), and connection capacity.
- **Processing throughput:** High rates of device requests, session events, and upgrade operations.
- **Concurrency management:** Controlling simultaneous upgrades and connection requests.
- **Reliability and fault tolerance:** Handling controller failures, offline devices, timeouts, failed downloads, and retries.
- **Data consistency:** Maintaining accurate device states and campaign progress across multiple controller instances.
- **Network impact:** Avoiding traffic spikes from connection requests, firmware downloads, and device reboots.
- **Monitoring and reporting:** Tracking device statuses and campaign results across millions of devices.

The objective is to evolve the current implementation into a scalable, distributed, and reliable system while preserving its existing firmware upgrade workflow and core logic.

2. Architecture and Horizontal Scalability

The current implementation provides the core firmware upgrade workflow, device state machine, and REST API. To support 5, 10, or 15 million devices, I would evolve it into a horizontally scalable, event-driven system while keeping the campaign orchestration logic largely unchanged.

The proposed architecture would consist of:

- **Multiple stateless controller instances** behind a **load balancer**, allowing independent scaling and fault tolerance.
- **A distributed message broker and worker pool** to asynchronously schedule upgrade operations, process device events, and manage retries.
- **A monitoring and reporting layer** providing real-time campaign progress, operational metrics, and historical reporting.

Campaign Scheduling and Controlled Rollout

Campaigns would be executed progressively through configurable rollout batches, rather than attempting to upgrade the entire target population simultaneously.

The existing `maxConcurrent` parameter would control the maximum number of device upgrades in progress at any time. Additional rollout parameters could define the initial batch size, subsequent batch sizes, and the interval between batches.

Campaigns could also support configurable **execution windows,** specifying **permitted days, hours, and time zones** for initiating firmware upgrades. This would allow operators to schedule upgrades during maintenance periods while avoiding peak traffic hours, such as executing the firmware upgrade in the night time with window from 23:00 PM to 05:00 AM as example.

This approach enables controlled rollout, limits the impact of firmware issues, and provides operators with greater control over campaign execution across large device populations.

3. Device Nudge and Upgrade Orchestration

I would support two ways of triggering firmware upgrades:

- **Event-based:** An optional `event` attribute in the campaign configuration would allow upgrades to be executed during a specified device session, such as `PERIODIC`, avoiding unnecessary connection requests.
- **Nudge-based:** If no event is specified, the controller would use the existing REST Connection Request mechanism to trigger device sessions.

The controller would manage upgrade operations according to the campaign's `maxConcurrent` limit, with controlled request rates, retries, and timeouts.

This provides flexibility in upgrade execution while reducing unnecessary device connections and network traffic.

3. Device Status Tracking and Campaign Reporting

The existing device state machine would remain the basis for tracking upgrade progress.

Each device's current status would be stored persistently, while important events and status changes (connection requests, download results, reboots, successes, and failures) would be recorded with timestamps and failure reasons.

The controller would provide campaign-level reports showing:

- Total targeted, pending, in-progress, succeeded, and failed devices.
- Success and failure rates.
- Retry counts and failure reasons.
- Campaign progress during execution and final results after completion.